

面试准备技能使用手册

本手册配合 `interview-skills-setup.md` 安装的技能与资料库使用。前提：已重启 Codex（或对应 AI 助手），使 `.agents/skills/` 下的新技能被加载。

一、已安装内容速查

名称	类型	用途	适用方向
<code>agent_java_offer</code>	资料库 (reference/)	Java 后端八股：MySQL、Redis、Kafka、并发、JVM、Spring、系统设计、算法、项目话术	Java 后端
<code>dotnet-core-azure-career-path</code>	资料库 (reference/)	C# 基础、ASP.NET Core、EF Core、Azure、DevOps 英文面试题库	C#/.NET Web
<code>wpf-questions</code>	资料库 (reference/)	C#/WPF 中文面试题（值类型、委托、GC、多线程、WPF 绑定等）	C# 桌面端
<code>interview-prep-generator</code>	技能	从简历 + JD 生成 STAR 故事库、行为面试题、回答话术	所有方向的行为面
<code>system-design-interview</code>	技能	生成系统设计题 → 模拟面试 → 按评分表复盘	系统设计
<code>requirements-prioritization-drill</code>	技能	5-10 分钟一轮的需求澄清 / 优先级练习（系统设计前 5 分钟热身）	系统设计
<code>interview-generation</code>	技能	生成 4 段式编码面试题（实现 / 算法 / 深挖 / 讨论）	编码面（默认 Python）
<code>interview-prep</code>	技能	综合出题：按岗位 / 公司 / 面试类型生成问题 + 回答要点	通用补充

二、开始前的准备

1. 重启 Codex，确认新技能已加载。
2. 准备两样输入材料：
 - 简历要点：建议整理成 `resume.md` 存到当前目录（按项目写清：做了什么、用了什么技术、量化结果）。
 - 目标岗位 JD：整理成 `jd.md`，或直接把 JD 文字粘贴到对话里。
3. 明确你的主攻方向（Java 后端 / C# .NET / 桌面端），下面按方向组合使用。

三、突击复习节奏（有基础 · 边找边面版）

核心思路：不按教材顺序学，而是以真实面试驱动——有基础的人缺的不是"再学一遍"，而是"查漏 + 表达 + 临场"。每次面试前针对性地补，面完当天复盘，把面试题变成自己的复习资料。

整个循环只有四件事：

环节	时机	做什么	用到的技能
1. 建弹药库（一次性，半天）	开始突击的第一天	整理 <code>resume.md</code> + <code>jd.md</code> ；生成 STAR 故事库和自我介绍脚本；让 AI 从 JD 反推考点清单	<code>interview-prep-generator</code> 、 <code>agent_java_offer</code>

环节	时机	做什么	用到的技能
2. 考前冲刺	每次面试前 1-2 天	只练 JD 反推的高频考点，不铺开刷；做 1 场全流程模拟	八股资料库 + 场景 B/C
3. 面试	真实面试	带精简面试包（自我介绍 + 10 个 STAR 故事 + 反问问题）	—
4. 面后复盘（当天）	每场面试结束后	记录被问到的题，分类「答好 / 一般 / 没答好」，补答案、补话术、更新错题本和故事库	场景 D

循环规律：**面 → 复盘 → 补短板 → 再面**。每场面试后错题本和故事库都会升级，下一场只会更强。

两点说明：

- 没有面试安排的碎片时间，跑每日 30 分钟三件套（场景 A）保持手感即可，不用刻意找大块时间刷题。
- 突击期 3-7 天足够形成第一版弹药库，之后是面试驱动的持续迭代，而不是线性学完。

四、每个技能 / 资料库怎么用

4.1 agent_java_offer —— Java 后端八股（资料库）

资料位于 `reference/agent_java_offer/docs/interview_prep/`，目录结构：

```
00_导航 / 01_AI / 02_后端 / 03_系统设计 / 04_算法 / 05_项目表达
02_后端 下按主题分章：01_MySQL 02_Redis 03_Kafka 04_并发
                        05_缓存与一致性 06_分库分表 07_JVM与GC 08_Spring 09_RPC 10_网络I/O 11_分布式事务
```

用法一：让 AI 按章节考你（推荐）

读取 `reference/agent_java_offer/docs/interview_prep/02_后端/01_MySQL/01_核心问答.md`，针对 MySQL 索引、事务隔离级别等高频面试题逐条考我：
先出题，我回答后再点评，给出标准答案和精炼的中文面试话术。

用法二：按目标岗位定制考点

我要面 3 年经验的 Java 后端岗，岗位要求里有高并发和微服务。
从 `reference/agent_java_offer/docs/interview_prep/02_后端/` 里挑出最可能考的 20 个问题，按「并发 → 缓存一致性 → Spring → 分布式事务」的顺序每天考我 5 题。

用法三：项目表达练习

读取 `reference/agent_java_offer/docs/interview_prep/05_项目表达/` 下的内容，根据我的项目经历，帮我设计「项目介绍 → 亮点 → 追问」的话术，并模拟面试官对我的项目连续追问 10 个问题。

4.2 C# / .NET 资料库 (dotnet-core-azure-career-path + wpf-questions)

读取 reference/dotnet-core-azure-career-path/csharp/index.md 和 aspnet-core/ 的索引页，按章节给我出 C# 面试题。我回答后对照资料里的参考答案点评，并给出更精炼的中文答题话术。

桌面端 (WPF) 单独练：

读取 reference/wpf-questions/README.md，重点考我 WPF 章节：绑定、依赖属性、线程模型、布局与控件，并对照资料给出标准回答和面试话术。

4.3 interview-prep-generator —— 行为面试 (STAR 故事库)

把简历要点和 JD 发给它，一次生成完整面试包：

使用 interview-prep-generator 技能。
我的简历在 resume.md，目标岗位 JD 在 jd.md。
请分析岗位要考察的核心能力，生成 10-12 个覆盖领导力 / 问题解决 / 协作 / 成就 / 失败成长的 STAR 故事，每个故事给出完整版 (2 分钟)、精简版 (60 秒) 和一句话版，并输出 "Tell me about yourself" 两分钟自我介绍脚本。

建议后续做三件事：

1. 把生成的故事用你自己的真实数据改准 (技能会留占位符，必须替换成真实项目和数字)。
2. 让 AI 模拟追问：用我的故事库，模拟面试官针对 "项目失败" 连续追问 5 轮，我回答后按 STAR 点评。
3. 面试前一天让它生成 "问题清单" 和 "红旗答案提醒"：基于 jd.md 生成 5 个我问面试官的问题和 3 个要避免踩的坑。

4.4 system-design-interview —— 系统设计模拟

三种模式：

模式一 (生成题)：使用 system-design-interview 技能，给我生成一道 45 分钟、适合 3 年经验 Java 后端的中等难度系统设计题。

模式二 (模拟面试)：用刚生成的 sd_*.md 开始模拟面试，你当面试官，我不会主动要提示，请按 5 个阶段推进 (需求 → 实体 → API → 高可用设计 → 深挖)。

模式三 (复盘)：我画完了，请读取我填好的 sd_*.md 内容，按评分表的四个维度 (问题导航 / 方案设计 / 技术深度 / 沟通表达) 给我打分和具体改进建议。

提示：该技能建议用 Obsidian + Excalidraw 画白板。不用 Obsidian 也没关系——你可以用文字 / 手绘图截图发给 AI，让它照样复盘。

4.5 requirements-prioritization-drill —— 需求澄清热身 (5 分钟一轮)

适合在正式系统设计练习前快速练 "前 5-10 分钟"：

使用 requirements-prioritization-drill 技能，给我连续跑 3 轮需求澄清练习，重点练：怎么提问、怎么把模糊题目收敛成 3 个功能需求 + 3 个非功能需求，并给我反馈。

4.6 interview-generation —— 编码面试题

⚠ 注意：这个技能默认生成 Python 题。Java 用户务必在指令里要求改写：

使用 interview-generation 技能，生成一道实现型编码题，风格选 mixed，带测试。请把题目和代码骨架改成 Java 17，并保持 4 段式结构（核心实现 → 扩展 → 深挖 → 讨论）。

它只负责出题，不负责判分；写完代码后你可以让 AI 对照检查：

这是我写的解法，请按题目的评价标准逐条检查：
覆盖哪些边界情况、有没有漏状态、复杂度如何，最后给出分数和改进建议。

4.7 interview-prep (tatat) —— 综合出题补充

质量一般，只作补充，用于考前快速过一遍：

使用 interview-prep 技能，按「Java 后端 / 3 年经验 / 技术面 + 行为面」生成 20 道题，每题说明考察意图、STAR 框架和回答要点。

五、组合使用场景

场景 A：每日 30 分钟（日常保持手感）

接下来 30 分钟做三件事：

1. 从 agent_java_offer 的 02_后端 里随机挑 3 道八股题考我（10 分钟）；
2. 用 requirements-prioritization-drill 跑 1 轮需求澄清（10 分钟）；
3. 用我的 STAR 故事库随机抽 1 道行为题考我（10 分钟）。

每题我回答后都给简短点评，不要长篇讲解。

场景 B：考前完整模拟（约 90 分钟）

模拟一场 90 分钟的 Java 后端全流程面试，按真实节奏来：

0-10 分钟：自我介绍（用我的自我介绍脚本）

10-40 分钟：技术面，从 agent_java_offer 出 5-8 道八股 + 追问

40-70 分钟：系统设计（用 system-design-interview 的题目，按 45 分钟节奏走）

70-85 分钟：行为面，用 interview-prep-generator 的 STAR 题库

85-90 分钟：让我问你 3 个问题

结束后给我一份总分和分项评语。

场景 C：面试前 1-2 天（临考冲刺）

我后天要面 [公司] 的 [岗位]，请综合 resume.md 和 jd.md 生成一份精简面试包：

1. 10 道最可能考的技术题（从 agent_java_offer 对应章节挑）+ 每题答题要点；

2. 5 道最高频行为题 + 对应 STAR 故事；
 3. 公司调研要点和 3 个反问问题；
 4. 一个 2 分钟自我介绍脚本。
- 今晚我先过一遍，明晚请按这套题模拟一场 45 分钟面试并打分。

场景 D：面后复盘（每场面试当天）

我刚面完 [公司] 的 [岗位]，这是我被问到的问题：

[把题目粘进来，或贴面试记录]

请帮我：

1. 每题标记「答好 / 一般 / 没答好」，并说明理由；
2. 对「没答好」的题，从 agent_java_offer（或对应资料库）找到章节，给出标准答案和精炼的中文话术；
3. 把答不好的点整理进错题本（存成 错题本.md），并列 3 个下次面试前必须再过一遍的点。

六、注意事项

1. **技能触发方式**：对话里直接点名技能名（如"使用 interview-prep-generator 技能"），或让场景自动匹配；如果发现技能没生效，检查是否已重启 Codex。
2. **interview-generation 默认是 Python**：Java 求职者必须显式要求转成 Java，否则练的方向不对。
3. **system-design-interview 的白板依赖 Excalidraw**：不想装 Obsidian 就用手绘图 + 截图，效果差别不大。
4. **资料库是参考答案，不是唯一答案**：特别是系统设计和架构题，AI 点评时允许存在合理的替代方案，重点学取舍思路而不是背方案。
5. **STAR 故事必须真实化**：技能生成的模板只是骨架，数字和项目细节务必替换成你自己的，否则追问一问就穿帮。
6. **许可与使用边界**：agent_java_offer 为 CC BY-NC 4.0，仅限个人复习使用，不要商业化转售；本目录内容均为本地文件，不涉及外传。
7. **所有技能都是"练习伙伴"，不是标准答案生成器**：八股答案最好自己先答一遍再对照，AI 的点评才有意义。
8. **复盘是进步最快的方式**：每场面试结束当天就复盘，别隔天；答得不好的题第二天立刻重练一遍，比刷十道新题有用。